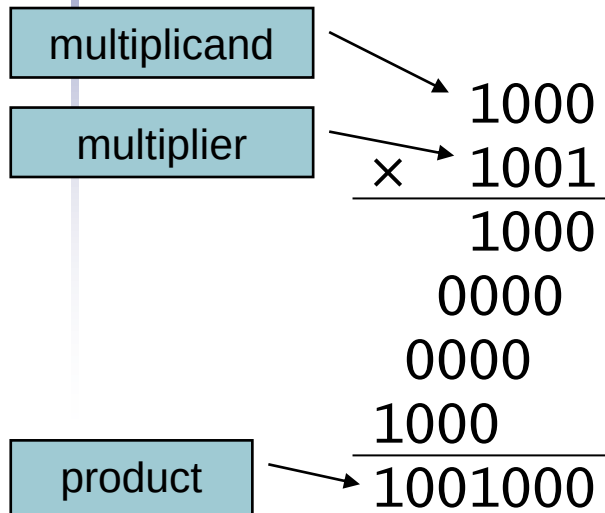


Computer Architecture

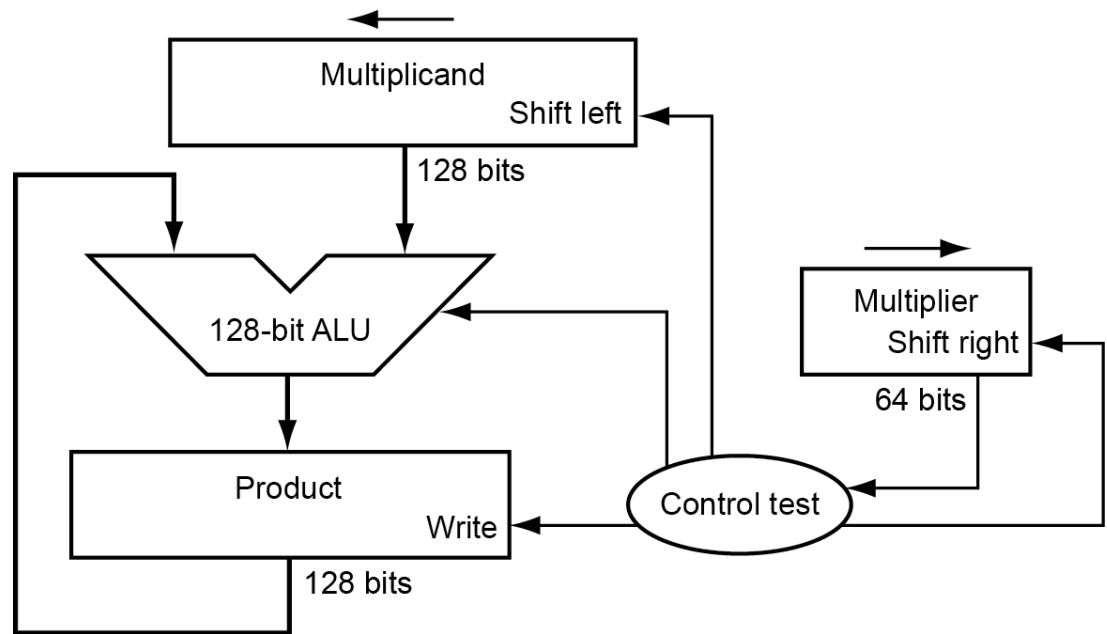
Lecture 6

Multiplication

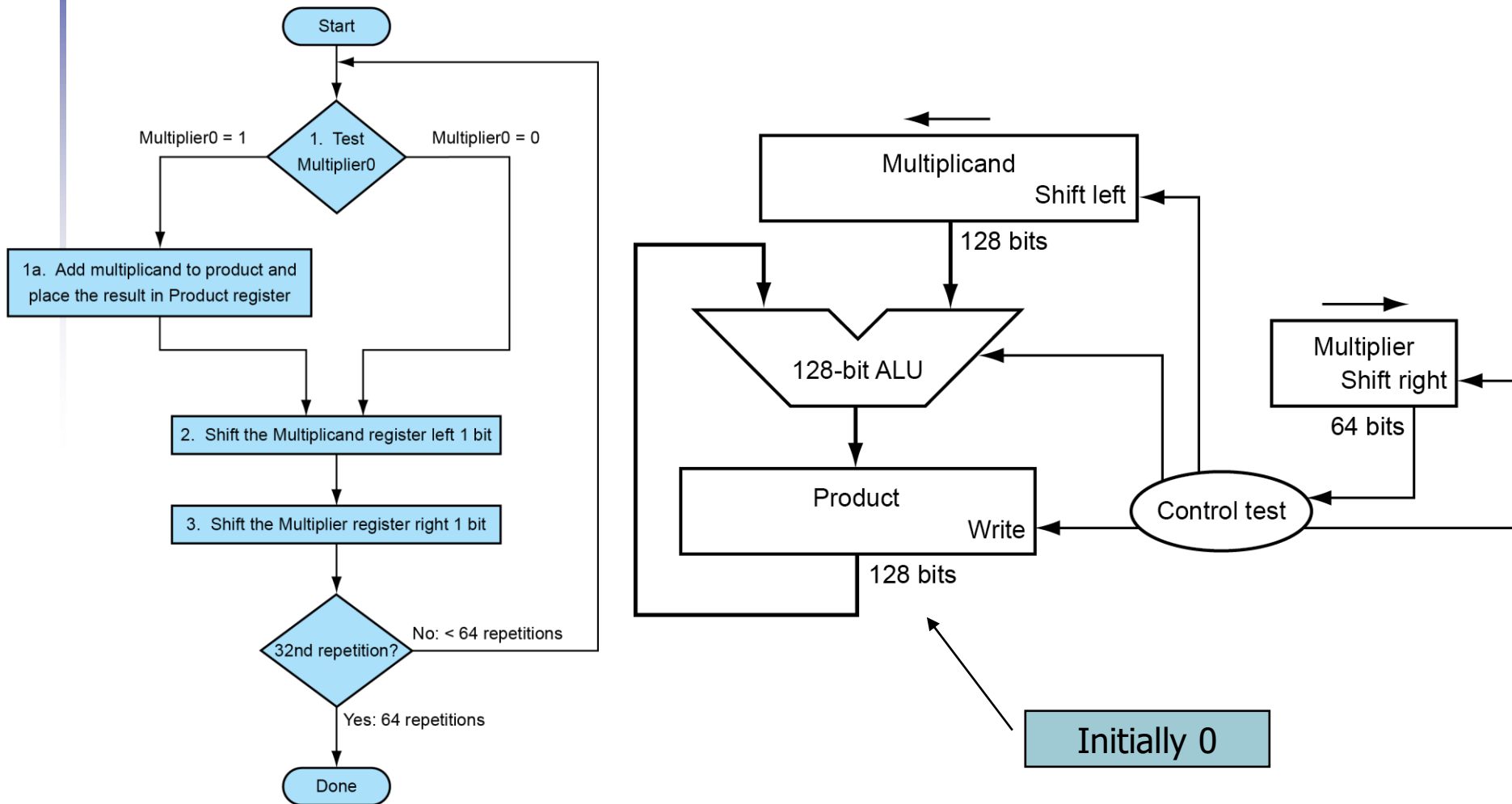
- Start with long-multiplication approach



Length of product is the sum of operand lengths

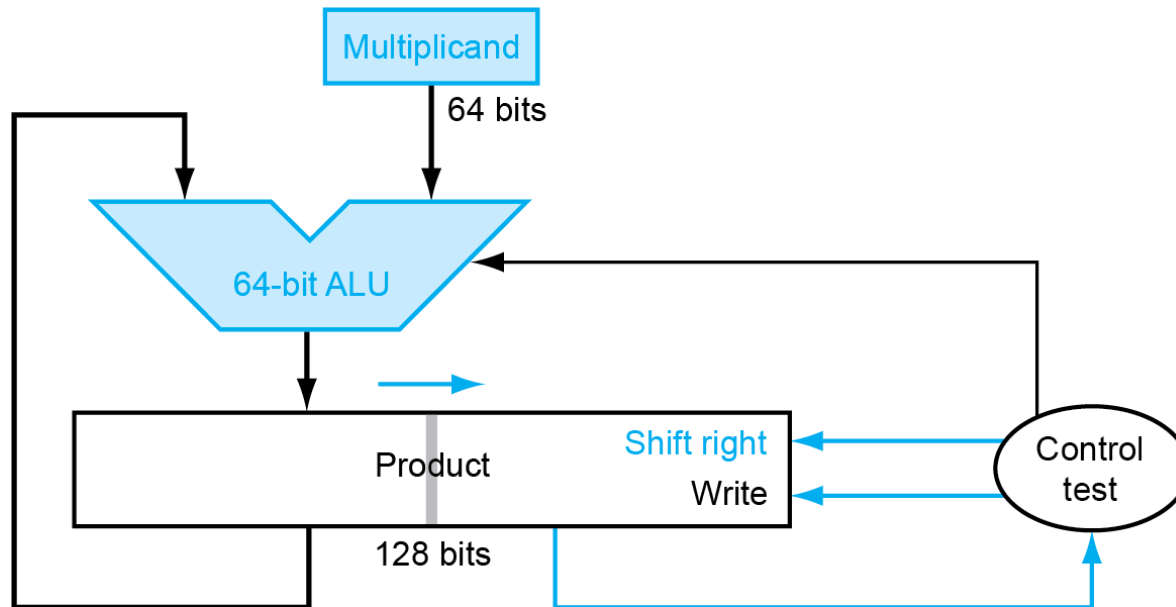


Multiplication Hardware



Optimized Multiplier

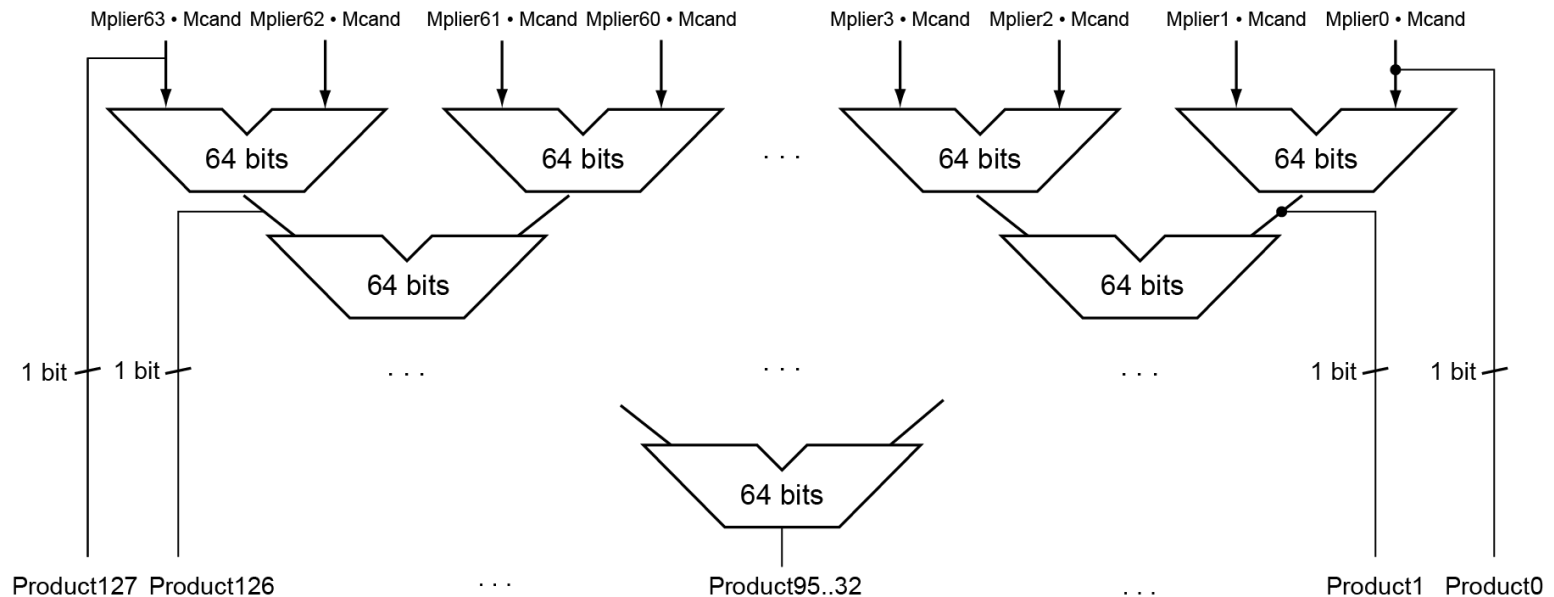
- Perform steps in parallel: add/shift



- One cycle per partial-product addition
 - That's ok, if frequency of multiplications is low

Faster Multiplier

- Uses multiple adders
 - Cost/performance tradeoff

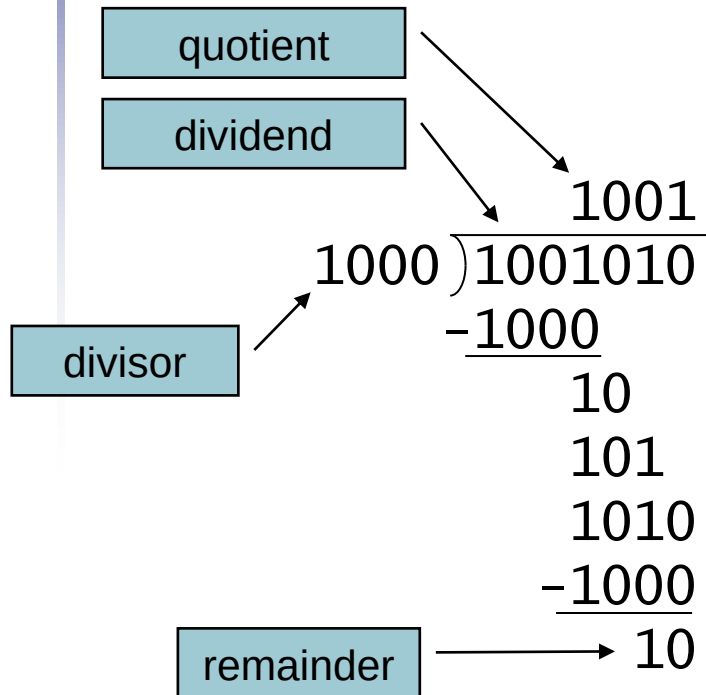


- Can be pipelined
 - Several multiplication performed in parallel

RISC-V Multiplication

- Four multiply instructions:
 - `mul`: multiply
 - Gives the lower 64 bits of the product
 - `mulh`: multiply high
 - Gives the upper 64 bits of the product, assuming the operands are signed
 - `mulhu`: multiply high unsigned
 - Gives the upper 64 bits of the product, assuming the operands are unsigned
 - `mulhsu`: multiply high signed/unsigned
 - Gives the upper 64 bits of the product, assuming one operand is signed and the other unsigned
- Use `mulh` result to check for 64-bit overflow

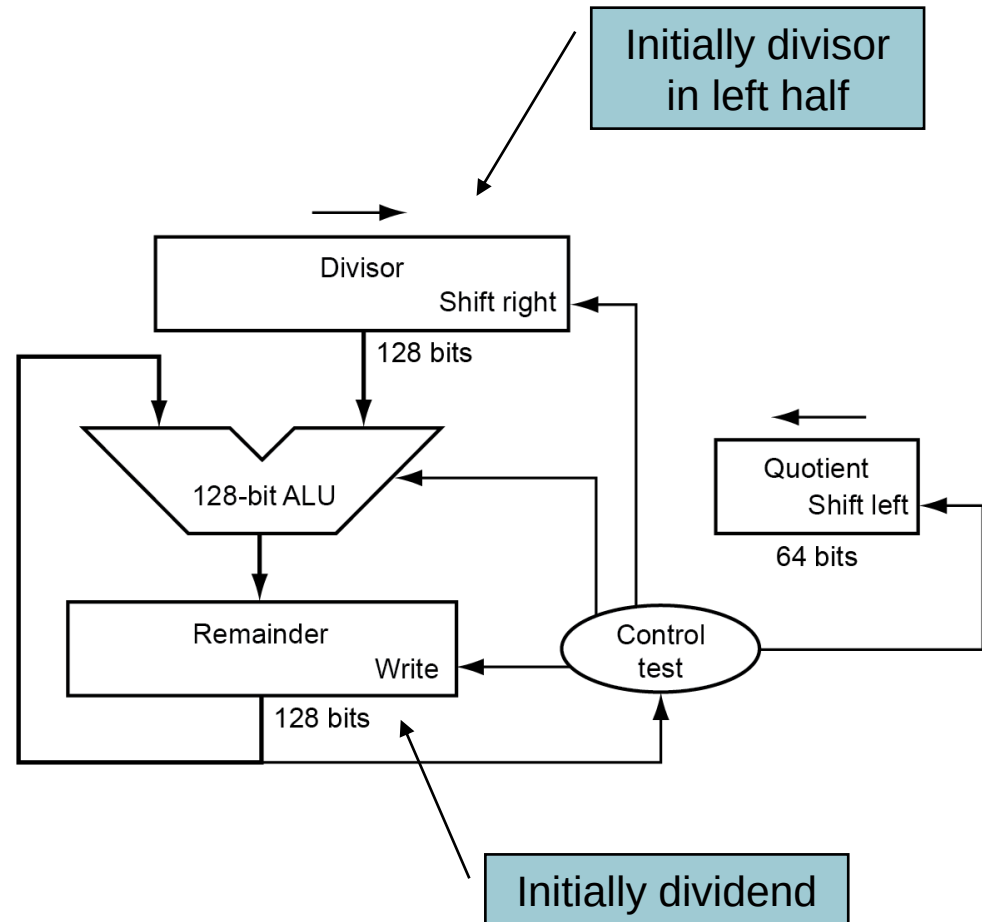
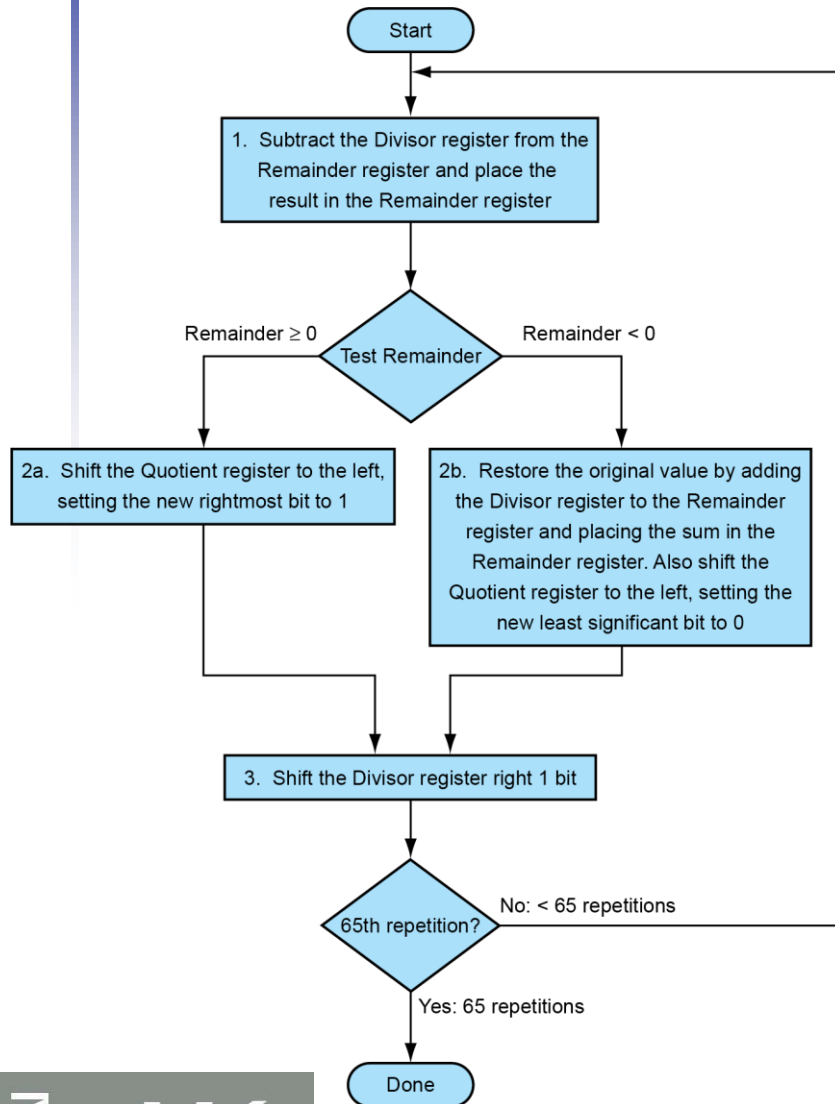
Division



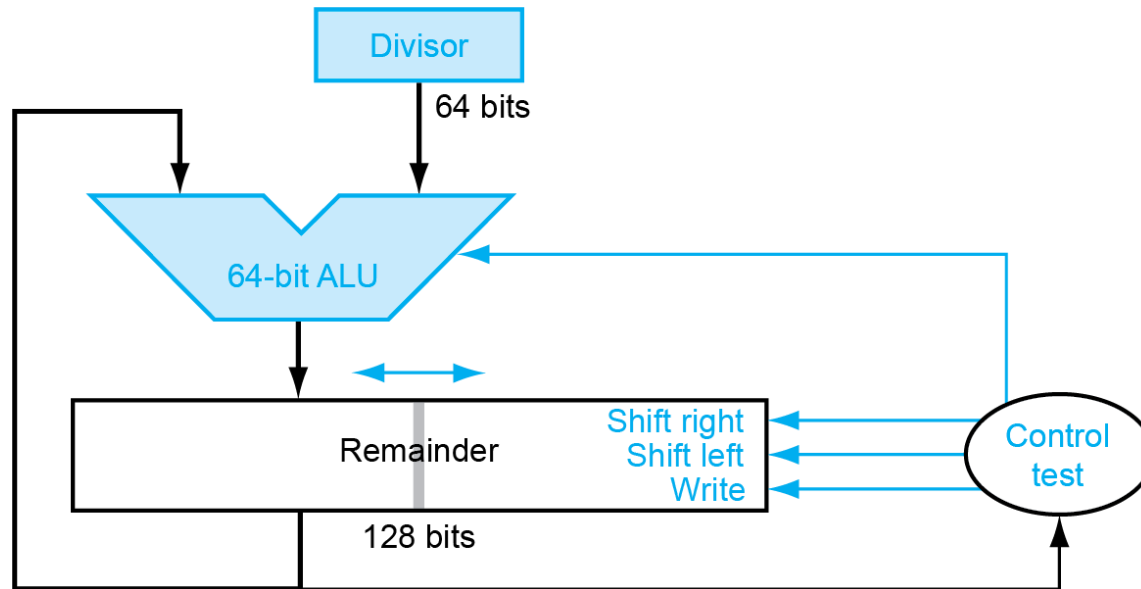
n-bit operands yield *n*-bit quotient and remainder

- Check for 0 divisor
- Long division approach
 - If divisor $\leq$ dividend bits
 - 1 bit in quotient, subtract
 - Otherwise
 - 0 bit in quotient, bring down next dividend bit
- Restoring division
 - Do the subtract, and if remainder goes < 0 , add divisor back
- Signed division
 - Divide using absolute values
 - Adjust sign of quotient and remainder as required

Division Hardware



Optimized Divider



- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
 - Same hardware can be used for both

Faster Division

- Can't use parallel hardware as in multiplier
 - Subtraction is conditional on sign of remainder
- Faster dividers (e.g. SRT division) generate multiple quotient bits per step
 - Still require multiple steps

RISC-V Division

- Four instructions:
 - div, rem: signed divide, remainder
 - divu, remu: unsigned divide, remainder
- Overflow and division-by-zero don't produce errors
 - Just return defined results
 - Faster for the common case of no error